



Desarrollo de API de un Sistema de Envíos.

Docente investigador: William Jose Morales Berrios (UNIVO, j.william03@hotmail.com)

Alumnos: Argueta Sánchez, Marilyn Gabriela (UNIVO, u20251105@univo.edu.sv), Benitez Machado, Steisy Gissel (UNIVO, u20230214@univo.edu.sv), Jiménez Arias, Marilyn Michelle (UNIVO, u20231085@univo.edu.sv), Rivera Leiva, Benjamín Franklin (UNIVO, u20250938@univo.edu.sv)

Resumen: Este proyecto consiste en el desarrollo de una API REST (el motor backend del sistema) para controlar y organizar el envío de paquetes en El Salvador. La plataforma centraliza la información de usuarios, empresas aliadas, pilotos y el estado de los paquetes en un solo lugar. Para construirlo, se utilizarán tecnologías modernas como ASP.NET Core Web API, SQL Server para la base de datos, Entity Framework Core y Swagger, que genera un manual digital para entender cómo funciona el sistema.

Abstract: This project focuses on developing a REST API to manage nationwide shipments in El Salvador. The system centralizes the administration of users, companies, drivers, and packages within a single backend platform. The solution will be built using ASP.NET Core Web API, SQL Server, and Entity Framework Core, incorporating Swagger for interactive endpoint documentation.

Palabras clave: API REST, ASP.NET Core, Swagger, SQL Server, Entity Framework Core, Gestión de Envíos. Backend, Arquitectura Onion.

I. JUSTIFICACION.

En El Salvador, muchas empresas de logística y tiendas en línea todavía controlan sus entregas de forma manual, es decir a papel y lápiz o usando hojas de cálculo en Excel. Este método tradicional suele causar problemas operativos: la información se pierde fácilmente, los estados de los paquetes no se actualizan a tiempo y es difícil organizar las rutas diarias de los repartidores.

Con el auge del comercio electrónico al automatizar estos procesos ya no es un lujo, sino que es una necesidad. Este proyecto nace para resolver ese problema mediante el desarrollo de una API REST funciona como el cerebro del sistema. Su objetivo es digitalizar todo el camino que recorre un paquete, permitiendo registrar la mercancía, asignarle un piloto encargado y saber con certeza en qué etapa se encuentra: si ya se recolectó, si está guardado en bodega, si va en camino, si ya se entregó con éxito o si hubo alguna devolución.

La ventaja principal es centralizar la información, así al guardar todo en una base de datos organizada, cualquier miembro del equipo puede consultar el estado de un envío en cuestión de segundos, eliminando la necesidad de hacer llamadas telefónicas constantes a los pilotos o buscar entre papeles. Esto mejora el trabajo interno de la empresa evitando fallas internas en el proceso y esto ofrece un servicio más transparente y confiable para el cliente final.

Finalmente, desde el lado académico, este proyecto nos permite como estudiantes poner en práctica herramientas

muy demandadas en el mundo laboral actual como lo es: ASP.NET Core para programar las reglas del negocio, SQL Server para el almacenamiento seguro de los datos, Entity Framework Core para conectar el código con la base de datos de forma eficiente, y Swagger para documentar el sistema de manera que otros desarrolladores puedan entenderlo y conectarlo fácilmente en el futuro.

II. OBJETIVOS.

Objetivo General.

Desarrollar un sistema backend (API REST) que centralice, organice y permita rastrear los envíos de paquetes a nivel Nacional.

Objetivo específico.

- **Crear los módulos de gestión (CRUD):** Desarrollar las funciones necesarias para registrar, consultar, actualizar y eliminar la información de usuarios, empresas y pilotos en el sistema.
- **Automatizar la asignación de entregas:** Permitir el registro digital de nuevos paquetes y vincularlos automáticamente con el piloto que se encargará de transportarlos.
- **Controlar el historial del paquete:** Implementar un sistema de estados dinámicos (recolectado, en bodega, en ruta, entregado, devolución) para saber exactamente dónde está cada envío.
- **Asegurar la disponibilidad de la información:** Garantizar que los datos se guarden de forma segura y ordenada para que el historial de cualquier paquete se pueda consultar de inmediato.
- **Facilitar la documentación técnica:** Integrar la herramienta Swagger para generar de forma automática un catálogo visual de todas las funciones del sistema, facilitando sus pruebas y futuras mejoras.

III. METODOLOGÍA.

La metodología aplicada en este proyecto posee un enfoque de desarrollo tecnológico y descriptivo. A través del análisis de los requisitos del sistema logístico (valores de entrada como datos de paquetes, pilotos y empresas), se obtiene el diseño arquitectónico del software. El tipo de investigación-desarrollo es de carácter no experimental aplicable, debido a que se toma como base un modelo arquitectónico de capas para software de servicios web, se implementa y se evalúan las respuestas operativas de la salida del sistema. El software se modela en función de las variables de entrada y salida del backend, las cuales son: las solicitudes de red del cliente (HTTP Requests), el procesamiento de la lógica de negocio y las respuestas del servidor (HTTP Responses con estados



200, 400, 500, etc.). Se toma como muestra técnica el flujo estándar de envíos postales y de comercio electrónico en el territorio de El Salvador, describiendo su operación mediante el consumo de servicios web.

IV. MARCO TEORICO.

Las arquitecturas orientadas a servicios son fundamentales para las plataformas de comercio electrónico modernas que exigen rapidez y disponibilidad de datos. El backend es el núcleo de la aplicación; a partir de los flujos de información y el volumen de datos que se desea procesar, se diseña el sistema que se implementará.

La API REST [1]. Proveer adecuadamente una interfaz de comunicación no solo significa hacerlo de forma eficiente y segura para los servidores y los datos; sino que también significa proveer la información en la cantidad, calidad y formato que el cliente necesita.

El formato de los datos se refiere principalmente al orden y estructura de los textos con los que se suministra esa información. En el desarrollo web moderno, las aplicaciones front-end (móviles o web) funcionan consumiendo datos en formato JSON (JavaScript Object Notation), y por lo tanto pueden ser alimentadas a través de un servidor backend cuya respuesta se mantiene estandarizada.

Por otra parte, la comunicación en la web utiliza el protocolo HTTP para funcionar. El 95% de los sistemas conectados a la red convencional en América Central operan bajo este protocolo a través de peticiones como GET, POST, PUT y DELETE. Los datos de los paquetes, empresas y pilotos se guardan de forma permanente en un motor de base de datos relacional por lo que se requiere de un componente intermedio, el ORM (como Entity Framework Core), que transforme, a través de mapeos lógicos, las tablas de la base de datos en objetos de código que el lenguaje de programación pueda entender fácilmente.

La Arquitectura Onion es un patrón de diseño de software propuesto para separar responsabilidades mediante capas concéntricas. Su objetivo principal es mantener la lógica de negocio independiente de tecnologías externas como bases de datos, frameworks o interfaces de usuario. Esta arquitectura mejora la mantenibilidad, escalabilidad y capacidad de prueba del sistema.

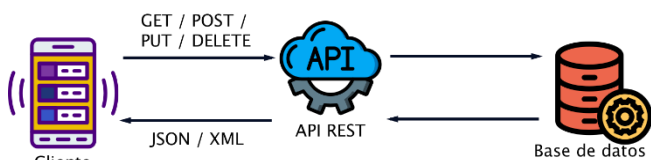


Fig. 1 : Modelo de una API REST.

Fuente: <https://dossetenta.com/wp-content/uploads/2024/03/R6qFq3n.png>

V. DESARROLLO DEL PROYECTO.

Diseño lógico del sistema

Para el desarrollo de la fuente de datos y la lógica de negocio que gestiona las operaciones de envío, se utilizó un diseño arquitectónico basado en el patrón de arquitectura en capas con ASP.NET Core Web API. El sistema utiliza varios componentes de software para estabilizar el flujo de información y regular las respuestas enviadas al usuario.

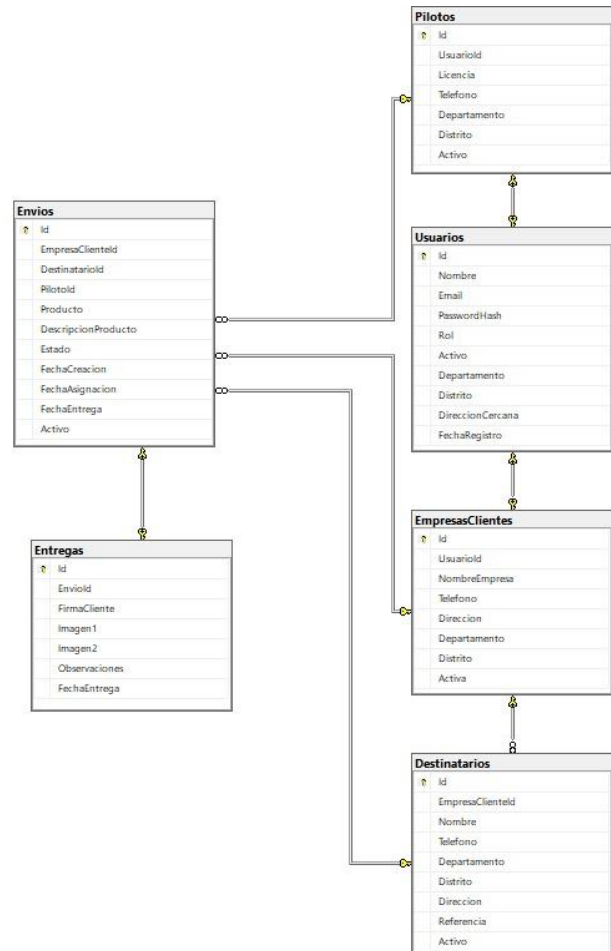


Fig. 2: Diagrama de Entidad-Relación del Sistema de Envíos.

Fuente: Elaboración propia.

El sistema utiliza una base de datos relacional compuesta por las entidades Usuarios, Pilotos, EmpresasClientes, Destinatarios, Envios y Entregas. La tabla Usuarios almacena la información general de acceso al sistema, mientras que Pilotos y EmpresasClientes se relacionan con ella para representar los distintos tipos de usuarios. Los destinatarios son asociados a una empresa cliente, y cada envío vincula a un destinatario, una empresa y un piloto responsable. Finalmente, la entidad Entregas almacena la evidencia de la recepción del paquete mediante firma, imágenes y observaciones.



Arquitectura Onion Implementada

La solución fue desarrollada utilizando el patrón de Arquitectura Onion, el cual divide el sistema en capas independientes para mejorar el mantenimiento y escalabilidad.

- API: contiene los Controllers encargados de recibir las solicitudes HTTP y exponer los endpoints mediante Swagger.
- Application: contiene los DTOs y servicios responsables de procesar la lógica de negocio.
- Domain: almacena las entidades principales del sistema, tales como Usuario, EmpresaCliente, Piloto, Destinatario, Envío y Entrega.
- Infrastructure: contiene el acceso a datos, repositorios, migraciones y configuración de Entity Framework Core.
- SQL Server: almacena permanentemente la información gestionada por la API.

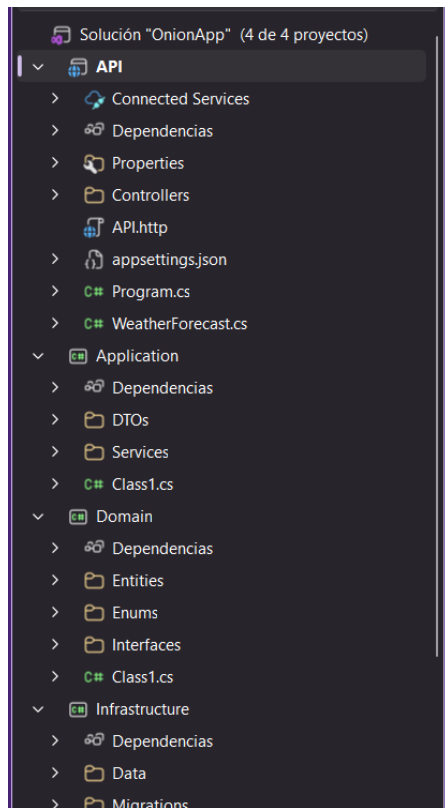


Fig. 3: Estructura del proyecto en Visual Studio. Fuente: Elaboración propia.

Funcionamiento de una solicitud HTTP en la API REST

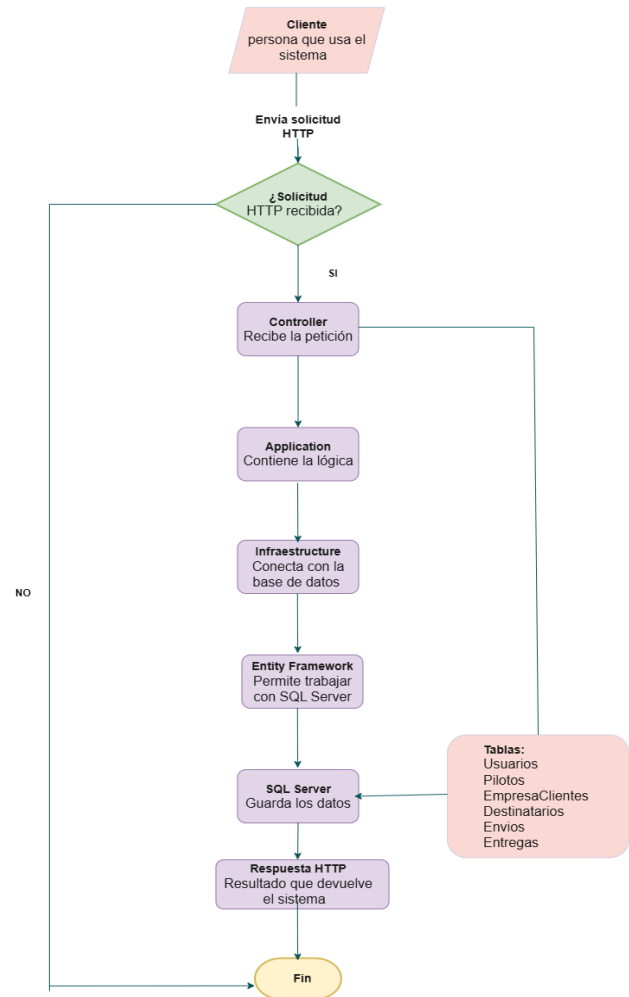


Fig. 4 Funcionamiento de una solicitud HTTP en la API REST
Fuente: Elaboración propia.

ENTIDAD	DESCRIPCIÓN
USUARIOS	Gestiona acceso al sistema
PILOTOS	Información de repartidores
EMPRESASCLIENTES	Empresas que realizan envíos
DESTINATARIOS	Personas que reciben paquetes
ENVIOS	Registro principal del envío
ENTREGAS	Evidencia de entrega

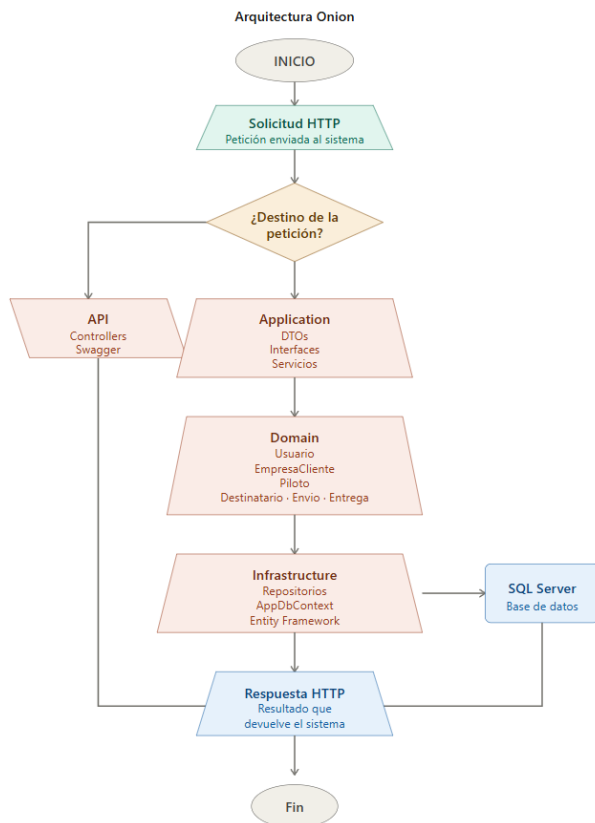


Fig. 5: Arquitectura Onion Implementada Fuente: Elaboración propia.

Funcionamiento del Backend:

- **Entrada de solicitudes (HTTP Requests):** El circuito de la API recibe las peticiones del usuario con datos en formato JSON para registrar empresas, pilotos o paquetes.
- **Controladores (Controllers [2]):** Actúan como los reguladores principales del sistema. Reciben la petición del cliente y derivan la orden a las capas internas correspondientes.
- **Entity Framework Core [3]:** Funciona como el puente regulador que permite variar y manipular los datos de la base de datos desde el código sin necesidad de escribir consultas SQL directas de forma manual.
- **Base de Datos SQL Server [4]:** Es el almacén permanente del sistema. Guarda de forma segura los registros de los pilotos, las empresas asociadas y los estados de las encomiendas.
- **Reglas de Validación (Capa de Negocio):** Actúan como filtros de protección del sistema. Evitan que ingresen datos corruptos o vacíos (como un paquete sin piloto asignado), protegiendo la integridad de la base de datos contra errores operativos.
- **Sistema de Estados Dinámicos:** Permite alterar el estado del paquete a medida que avanza en la ruta

física, variando entre los valores: Recolectado, En Bodega, En Ruta, Entregado y Devolución.

- **Swagger [5]:** Es el voltímetro del desarrollador; se alimenta de los endpoints construidos para mostrar una interfaz gráfica interactiva que permite medir, probar y visualizar el comportamiento en tiempo real de la API.

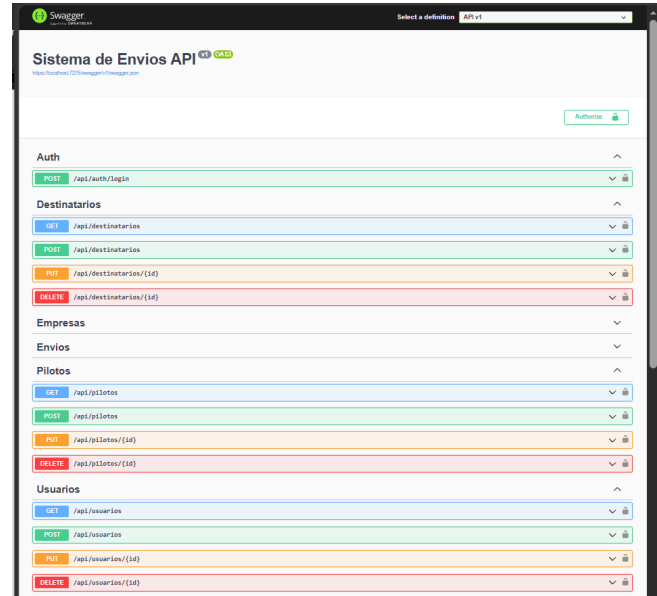


Fig. 6: Interfaz Swagger de la API de Envíos. Fuente: Elaboración propia.

Se implementaron DTOs para optimizar las respuestas de la API y evitar el envío innecesario de información.

Flujo Operativo del Sistema

El proceso inicia con el registro de la empresa cliente, el destinatario y la información general del paquete. Posteriormente se asigna un piloto encargado de la entrega y se validan los datos antes de almacenarlos en la base de datos.



Registro del Paquete

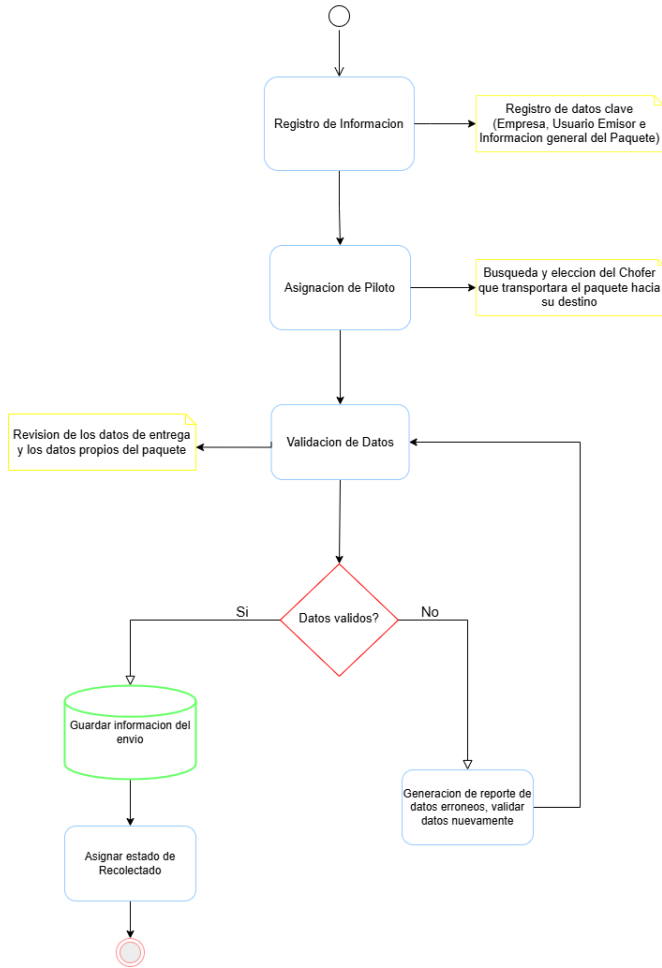


Fig. 7: Diagrama Registro del Paquete. Fuente: Elaboración propia.

Seguimiento de Envíos

Una vez registrado, cada envío pasa por diferentes estados operativos: Recolectado, En Bodega, En Ruta, Entregado o Devolución. Estos estados permiten conocer en todo momento la ubicación y situación actual del paquete.

Seguimiento del Paquete

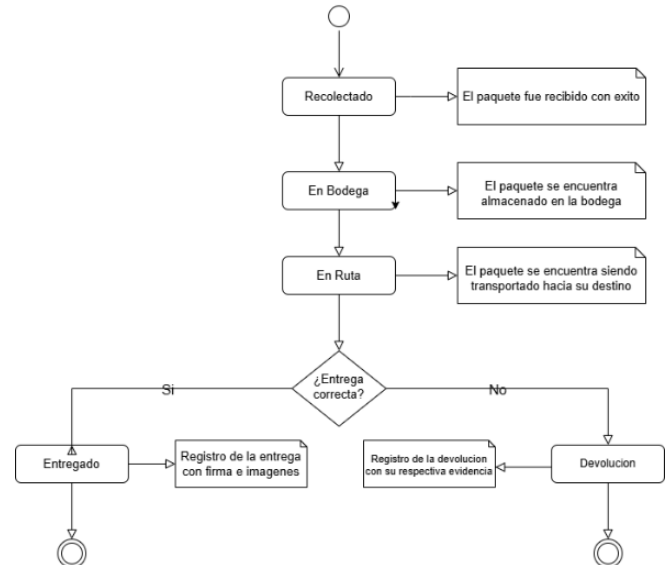


Fig. 8: Diagrama Seguimiento del paquete. Fuente: Elaboración propia.

Gestión de Usuarios y Seguridad

El sistema administra usuarios mediante credenciales de acceso almacenadas de forma segura. Cada usuario posee un correo electrónico, contraseña y rol específico dentro del sistema. La información de autenticación se encuentra almacenada en la tabla Usuarios de la base de datos.

Resultados e Implementación del Sistema

Debido a que los datos de las consultas complejas (como amarrar un paquete con su piloto y empresa en una sola vista) podían saturar el rendimiento, se implementó el uso de DTOs (*Data Transfer Objects*) a la salida de las respuestas de la API. Estos objetos optimizan la transferencia de carga de información hacia el exterior. A continuación, se describen los pasos seguidos para la construcción y diseño del software:

- **Paso I. Pruebas de concepto en memoria (Mocking):** Se procedió a ensamblar la lógica del circuito en código utilizando listas en memoria para probar el funcionamiento de la API, verificando la correcta asignación de paquetes a los pilotos antes de conectar la base de datos real.
- **Paso II. Diseño e Impresión del Modelo de Datos:** Se diseñó el diagrama de entidad-relación (la placa base de los datos) utilizando herramientas de modelado de software. Posteriormente se ejecutaron las *Migrations* para "imprimir" y estructurar de manera física las tablas en SQL Server.



- **Paso III. Grabado y Conexión de Capas:** Se configuraron las cadenas de conexión (*Connection Strings*) en el archivo de configuración del proyecto, permitiendo que la API REST lograra conectarse de forma fluida y directa con el motor de base de datos.
- **Paso IV. Pruebas y Ajustes Finales:** Al realizar las solicitudes finales, la salida de la API dio respuestas exitosas en milisegundos. Durante las pruebas, se detectó que las respuestas largas retrasaban el sistema, por lo que se ajustaron los objetos de transferencia de datos para optimizar la velocidad, logrando que el backend devuelva la trazabilidad completa del envío de manera inmediata y limpia.

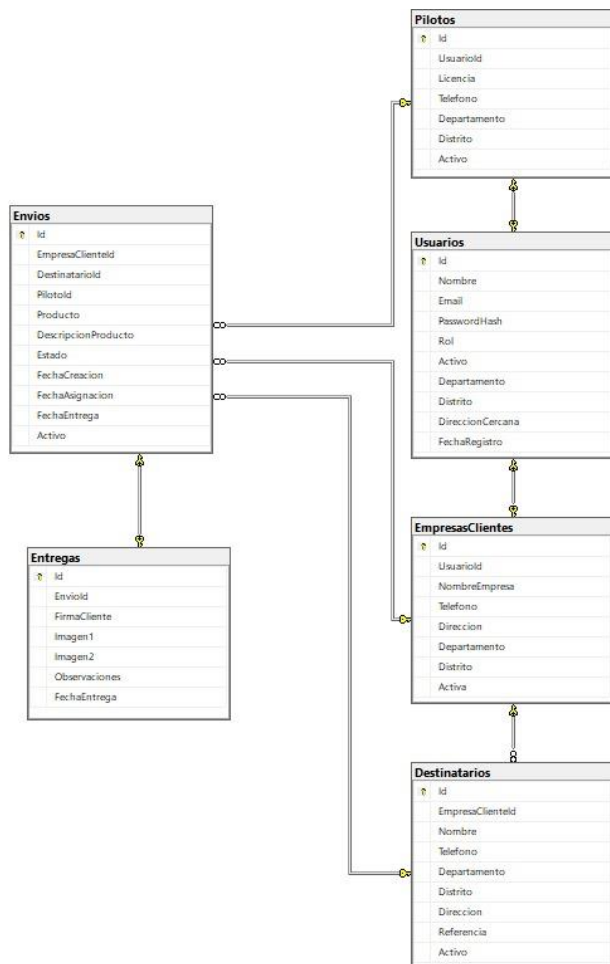


Fig. 9: Evidencia de relaciones entre tablas SQL Server.
Fuente: Elaboración propia.

La base de datos fue creada utilizando migraciones de Entity Framework Core. Las relaciones entre usuarios, empresas clientes, destinatarios, envíos, pilotos y entregas permiten mantener la integridad de la información durante todo el ciclo de vida de un paquete.

VI. CONCLUSIONES.

- Se desarrolló una API REST capaz de administrar empresas clientes, pilotos, destinatarios y envíos mediante una arquitectura organizada por capas.
- La utilización de Entity Framework Core permitió simplificar el acceso a datos y mantener sincronizada la aplicación con SQL Server.
- La implementación de estados dinámicos mejoró la trazabilidad de los paquetes durante cada etapa del proceso logístico.
- La Arquitectura Onion facilitó la separación de responsabilidades entre capas, mejorando la mantenibilidad y escalabilidad del sistema.

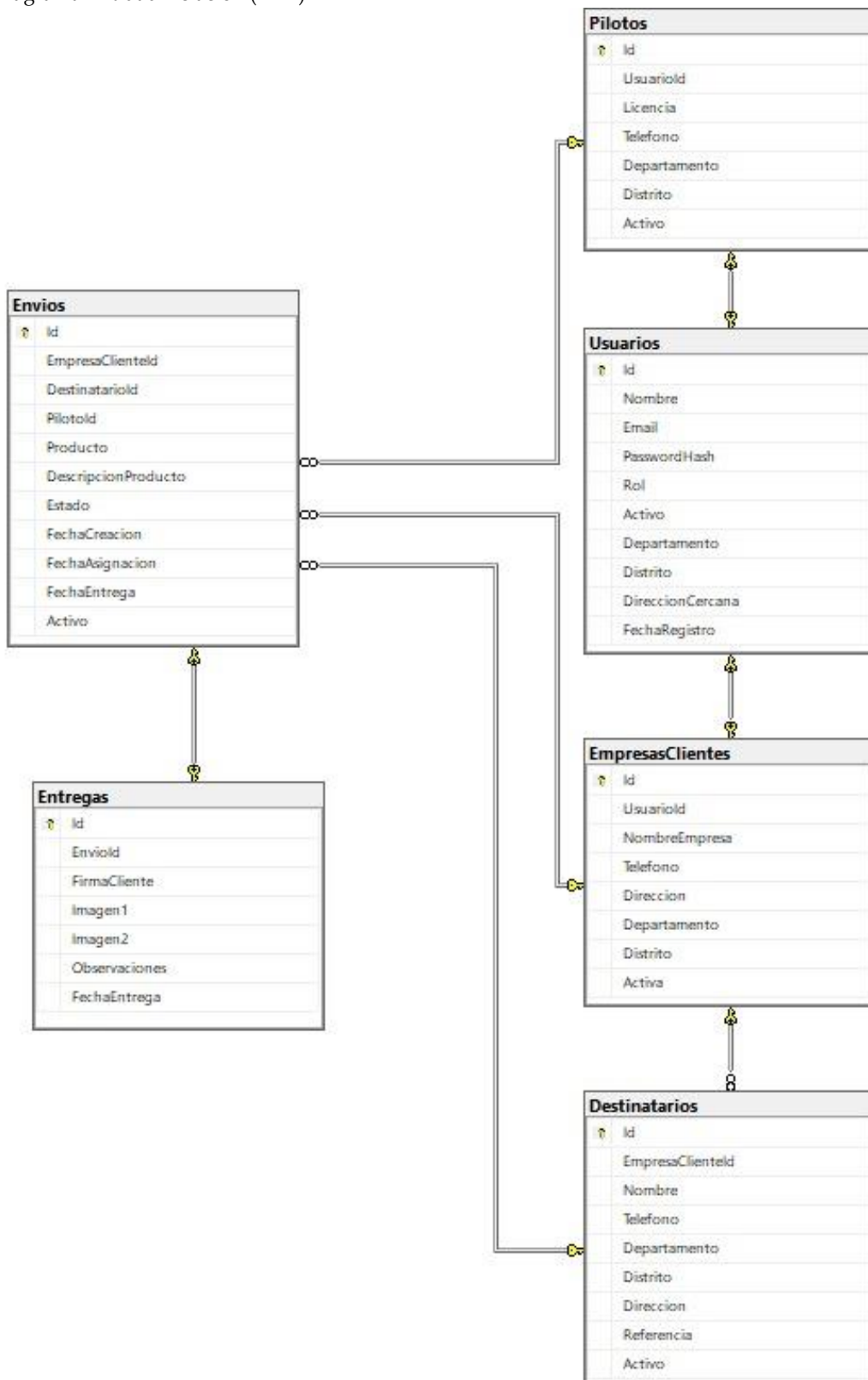
VII. BIBLIOGRAFIA.

VIII.

- [1] R. T. y. T. R. N. Fielding, «University of California, Irvine,» [En línea]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. [Último acceso: 25 Mayo 2026].
- [2] Microsoft, «Microsoft Learn - Tutorial: Creación de una API web con ASP.NET Core,» 2026. [En línea]. Available: <https://learn.microsoft.com/es-es/aspnet/core/tutorials/first-web-api>. [Último acceso: 25 Mayo 2026].
- [3] Microsoft, «Microsoft Learn - Inicio rápido de EF Core: Introducción a Entity Framework Core,» 2026. [En línea]. Available: <https://learn.microsoft.com/es-es/ef/core/get-started/overview/first-app>. [Último acceso: 25 Mayo 2026].
- [4] Microsoft, «Microsoft Learn - Instancias de características y motor de base de datos de SQL Server,» 2024. [En línea]. Available: <https://learn.microsoft.com/es-es/sql/database-engine/configure-windows/sql-server-express-blog>. [Último acceso: 25 Mayo 2026].
- [5] Microsoft, «Microsoft Learn - Documentación de la API web de ASP.NET Core con Swagger / OpenAPI,» 2025. [En línea]. Available: <https://learn.microsoft.com/es-es/aspnet/core/tutorials/web-api-help-pages-using-swagger>. [Último acceso: 25 Mayo 2026].

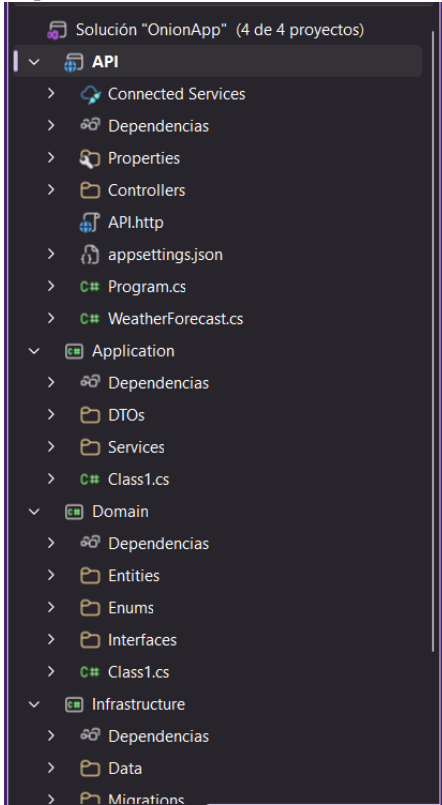


ANEXOS.
ANEXO I.
Diagrama Entidad-Relación (DER)

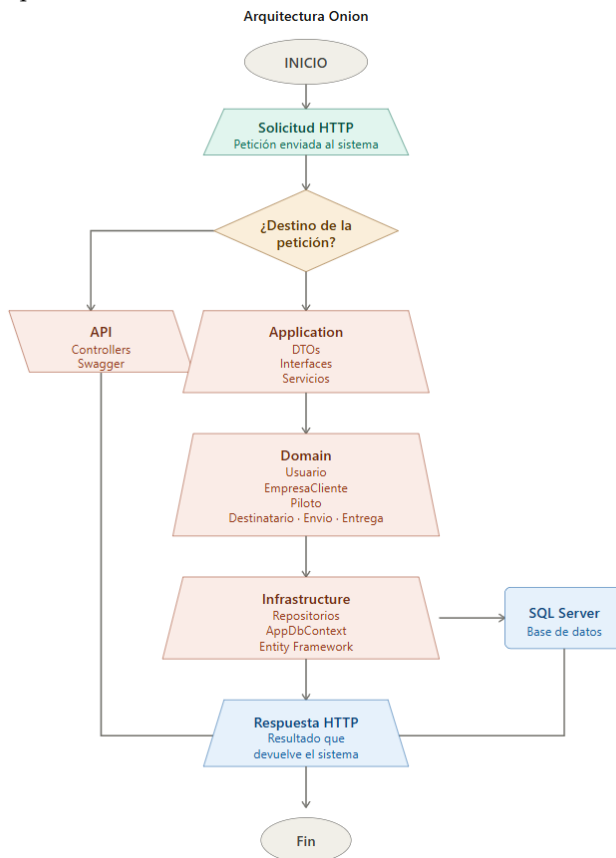




ANEXO II. Captura Visual Studio.

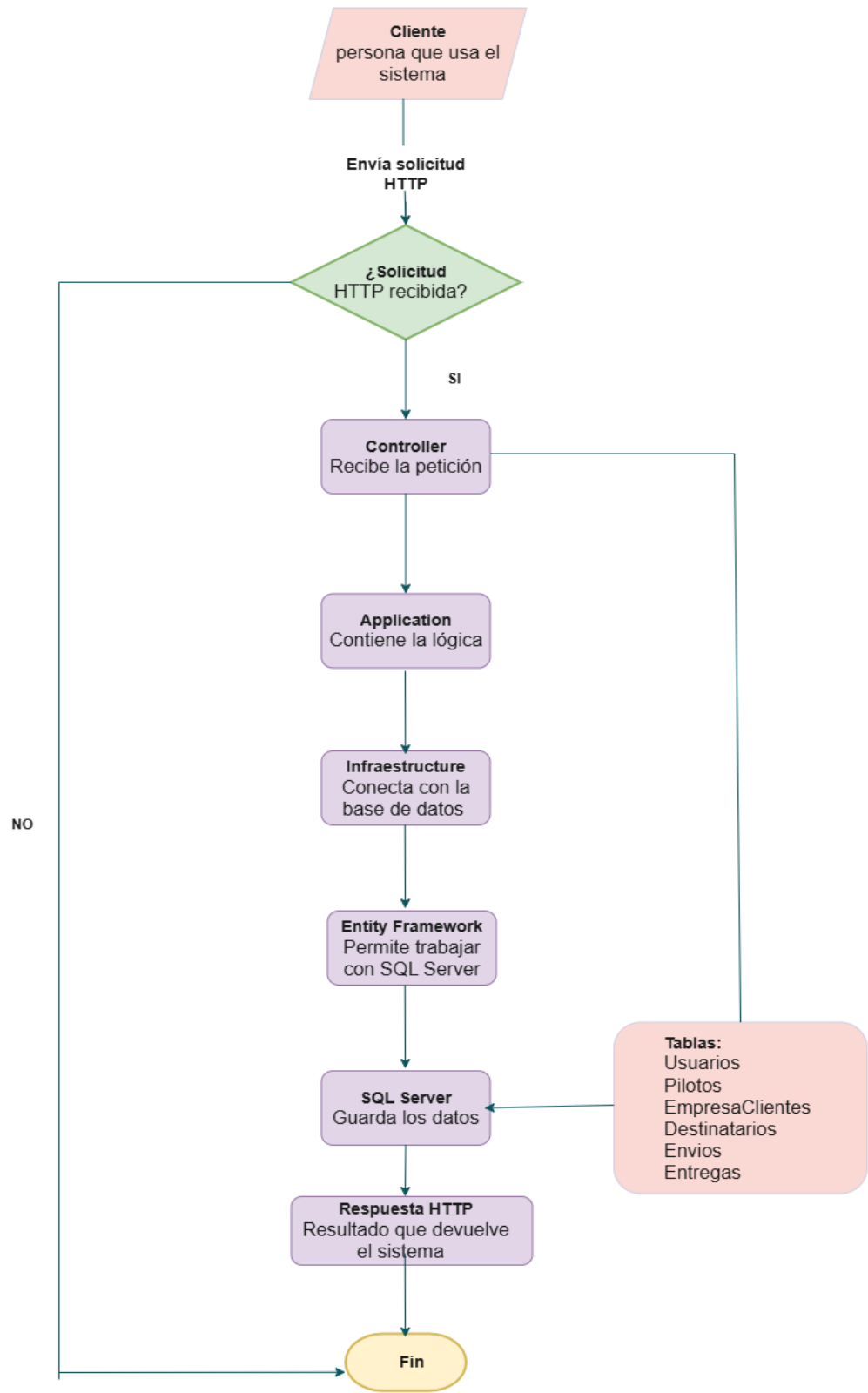


ANEXO III. Arquitectura Onion.



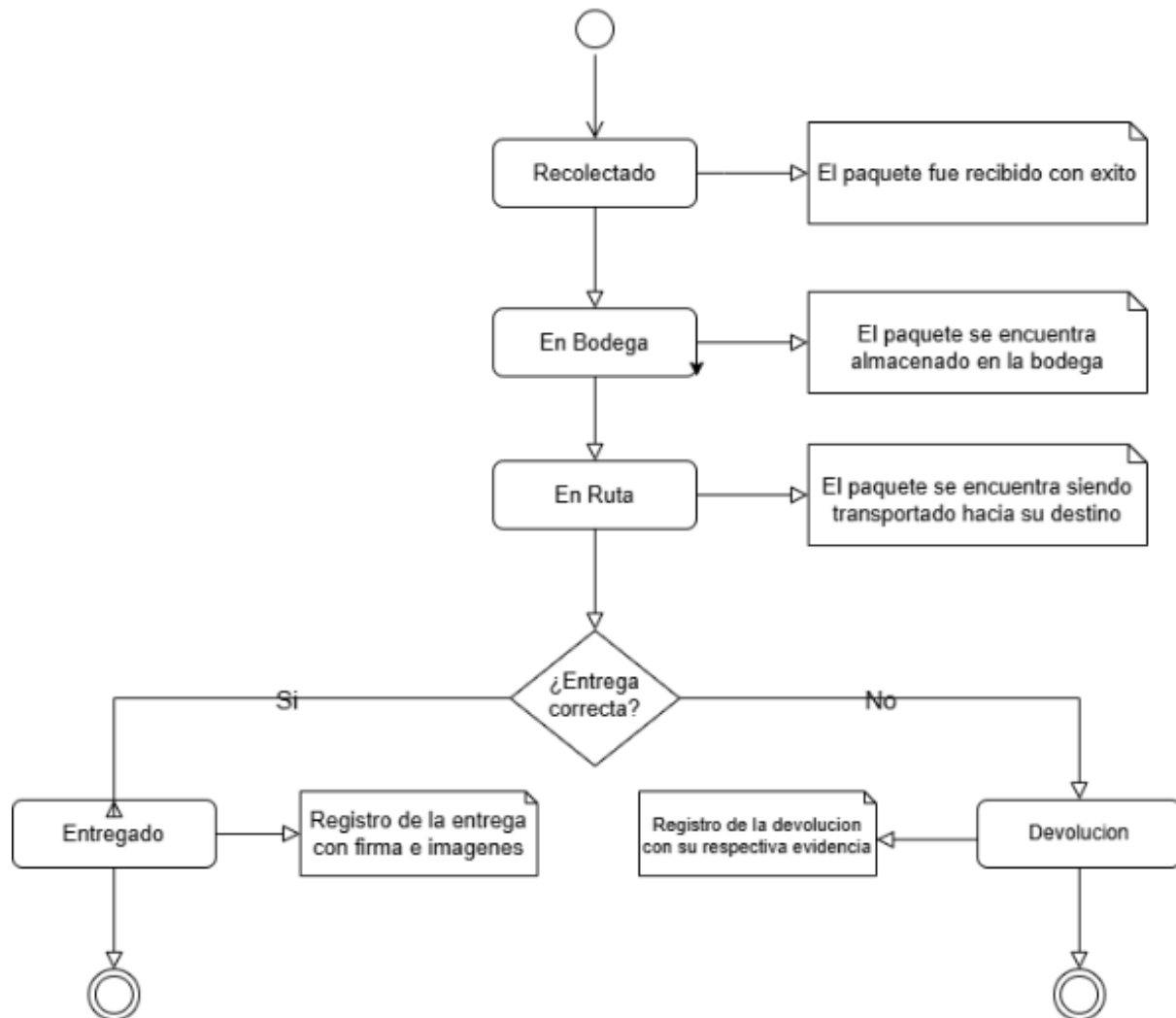


ANEXO IV.
Funcionamiento HTTP.





Seguimiento del Paquete





ANEXO VI
Swagger.

Swagger
Integrate with SMARTBEAR

Select a definitionAPI v1

Sistema de Envios API v1 OAS3
<https://localhost:7275/swagger/v1/swagger.json>

Authorize

Auth

POST

/api/auth/login

Destinatarios

GET

/api/destinatarios

POST

/api/destinatarios

PUT

/api/destinatarios/{id}

DELETE

/api/destinatarios/{id}

Empresas

Envios

Pilotos

GET

/api/pilotos

POST

/api/pilotos

PUT

/api/pilotos/{id}

DELETE

/api/pilotos/{id}

Usuarios

GET

/api/usuarios

POST

/api/usuarios

PUT

/api/usuarios/{id}

DELETE

/api/usuarios/{id}